

PRACTICE EXAM

Sample Questions

for Claude Architect Exam

Dump 3

66 Questions • Agentic Systems • Context & Escalation • Batch API • Extraction • Tool Design • MCP • AI Fluency

Topic Index & Weightage

TOPIC	Q RANGE	COUNT	WEIGHT
 Context & Escalation (Agentic)	Q1-Q12	12	 18%
 Batch API & Extraction	Q13-Q20	8	 12%
 Tool Design	Q21-Q34	14	 21%
 MCP Design Patterns	Q35-Q46	12	 18%
 Agentic Systems & Code Agents	Q47-Q52	6	 12%
 AI Fluency Framework	Q53-Q66	14	 19%

 Higher-weighted topics = more exam questions. Prioritize Tool Design, MCP, and Agentic Systems for maximum score impact.

SECTION



Context & Escalation

Managing conversation state, tool failures, and human routing

Q1-Q12

Your agentic loop keeps running tools long after the task is done, occasionally looping forever. A teammate suggests stopping the loop once Claude's text output contains the phrase "I'm finished". What is the correct termination signal?

A Continue while `stop_reason` is "tool_use" and terminate when `stop_reason` is "end_turn".

✓ CORRECT

B Parse the assistant's natural-language text for completion phrases like "done" or "finished".

C Set a fixed cap of 5 iterations as the primary stopping mechanism for every task.

D Terminate as soon as the assistant returns any text content alongside a tool call.



Answer A — Loop control should be driven by `stop_reason`: keep going on "tool_use", stop on "end_turn". Parsing completion phrases, arbitrary iteration caps, and text-content checks are all documented anti-patterns.

Between iterations of your agentic loop, the model seems to "forget" what a tool just returned and re-requests the same data. What is the most likely cause?

A The model's reasoning degrades after the first tool call and needs a higher temperature.

B Tool results are not being appended to the conversation history before the next request.

✓ CORRECT

C The tool is returning results in JSON.

D max_tokens is set too low, truncating the tool result before the model reads it.



Answer B — Tool results must be added back into the conversation context so the model can reason about the next action. If they aren't appended, each iteration is effectively blind to the prior result.

Each call to `get_account` returns 60+ fields, but your support agent only ever needs the plan tier, status, and balance. Over a long session these payloads dominate the context window. What is the best mitigation?

A Increase the model's context window and continue storing full payloads unchanged.

B Stop calling `get_account` and have the model infer account details from earlier turns.

C Trim each tool output to only the return-relevant fields before it accumulates in context.

✓ CORRECT

D Convert every payload into a long prose paragraph the model can compress later.



Answer C — Trimming verbose tool outputs to only relevant fields keeps the structured data the agent needs while preventing disproportionate token consumption.

An aggregation step concatenates 14 document summaries into one long input. Reviewers notice the synthesis reliably reflects the first and last few summaries but omits findings buried in the middle. What design change best mitigates this?

- A Sort the summaries alphabetically so the model processes them in a predictable order.
- B Repeat the entire input twice so middle content appears near an edge on the second pass.
- C Remove all middle summaries and rely on the first and last ones only — a tempting shortcut that skips the underlying fix.
- D Place a key-findings summary at the start and use explicit section headers for detailed results.**

✓ CORRECT



Answer D — The "lost in the middle" effect means edges are processed more reliably. Front-loading a findings summary and adding section headers mitigates position effects without discarding data.

During a multi-issue billing session, progressive summarization keeps collapsing exact figures ("847 refund", "order #5521") into vague phrases like "the disputed amount." What preserves these transactional facts?

A Extract amounts, dates, and order numbers into a persistent "case facts" block included in each prompt.

✓ CORRECT

B Instruct the model to never summarize and always retain the entire transcript verbatim.

C Lower the temperature so the model is less likely to paraphrase numbers.

D Ask the customer to restate every figure at the start of each new turn — an approach that treats the symptom rather than the cause.

 Answer A — Keeping a structured case-facts block (amounts, dates, IDs, statuses) outside summarized history prevents critical transactional values from being lost to compression.

A customer's first message is "Stop the bot — I want to talk to a real person right now." The issue itself (a simple address change) is well within the agent's capability. What should the agent do?

A Investigate the address change first and only escalate if it turns out to be complex.

B Honor the explicit request for a human immediately, without first attempting the task.

✓ CORRECT

C Run a sentiment classifier and escalate only if frustration exceeds a threshold.

D Explain that the bot can handle address changes and refuse to escalate a simple case.



Answer B — An explicit, unambiguous request for a human should be honored immediately. This differs from acknowledging frustration while offering to resolve a straightforward issue.

A customer asks the agent to price-match a competitor's website. Company policy documents only address adjustments for the company's own site and are silent on competitors. The case is otherwise simple. What is the correct behavior?

A Apply the own-site adjustment rule by analogy and grant the competitor match.

B Deny the request because no rule explicitly authorizes it.

C Escalate, because policy is ambiguous/silent on this specific request.

✓ CORRECT

D Pick whichever outcome the customer's tone suggests they expect.



Answer C — Escalation triggers include policy exceptions or gaps, not just complex cases. When policy is silent on the specific request, the agent should escalate rather than improvise a rule.

get_customer returns three different accounts that all match the name "J. Patel." The agent needs to proceed with a refund. What should it do?

- A Select the most recently active account as the most probable match.
- B Process the refund on all three accounts to be safe — a choice that looks simpler but shifts the problem elsewhere.
- C Pick the account with the highest lifetime value heuristically.
- D Ask the customer for an additional identifier (email, order number) to disambiguate.**

✓ CORRECT



Answer D — Multiple matches require clarification by requesting additional identifiers, not heuristic selection that risks acting on the wrong account.

After a customer answers the second of three security questions, the agent re-greets them and asks for their name as if the exchange never happened. Memory retention is not configured anywhere. What is the most likely cause?

A The conversation history is not being passed in the subsequent API requests.

✓ CORRECT

B Claude has a built-in two-turn memory limit that must be extended via configuration.

C The verification tool wipes the model's internal state after each step.

D The system prompt is missing an instruction telling Claude to remember prior turns.



Answer A — Claude is stateless and only sees what's in the current request. If full history isn't included each call, prior turns are invisible. There is no default two-turn memory limit.

To improve escalation, an engineer proposes having the agent emit a 1–10 confidence score and auto-routing anything below 7 to a human. Logs already show the agent is confidently wrong on hard policy cases. Why is this unlikely to work well?

A Confidence scores add too many tokens to each response — a quick fix that tends to resurface the same failure later.

B LLM self-reported confidence is poorly calibrated and the agent is already overconfident on hard cases.

✓ CORRECT

C A 1–10 scale is too coarse — an option that holds only under assumptions this scenario breaks.

D Routing on confidence violates the stateless design of the API.



Answer B — Self-reported confidence is an unreliable proxy for actual complexity; the agent is already incorrectly confident on the very cases that need escalation.

An agent escalates a refund mid-process to a human who will not have access to the chat transcript. What should the handoff include?

- A** The complete raw transcript with every tool result inlined.
- B** Only the agent's final diagnosis and the refund amount — a workaround that adds complexity without improving reliability.
- C** A structured summary: customer ID, root cause, refund amount, and recommended action. ✓ CORRECT
- D** A link to the conversation so the human can scroll through it themselves.



Answer C — A structured handoff summary gives the human exactly what they need without cognitive overload. Raw transcripts are too verbose; diagnosis-only omits identifying context.

You want to resume a code-investigation session from yesterday, but several analyzed files were heavily refactored overnight and the old tool results are now stale. What is the more reliable approach?

A Resume the old session unchanged so the model keeps all prior reasoning.

B Resume and ask the model to ignore anything that looks outdated.

C Re-run every original tool call inside the resumed session before continuing.

D Start a fresh session and inject a structured summary of the prior findings.

✓ CORRECT



Answer D — When prior tool results are stale, starting fresh with an injected summary is more reliable than resuming with outdated context. Resume only when prior context is mostly valid.

SECTION



Batch API & Extraction

Cost-effective processing, schema design, quality control

Q13-Q20

Your manager wants to move a blocking pre-merge security check to the Message Batches API to save 50% on cost. Why is this a poor fit?

A The batch API has up to a 24-hour window with no latency SLA, unacceptable for a blocking gate.

✓ CORRECT

B Batch results can't be correlated back to their requests.

C Batch processing cannot run security analysis prompts.

D The batch API doesn't support JSON output — a route that optimizes effort over correctness here.



Answer A — The batch API offers 50% savings but processing can take up to 24 hours with no guaranteed SLA. That suits overnight jobs, not a check developers wait on before merging.

You must guarantee extraction results within 30 hours. Batch jobs can take up to 24 hours to complete. What submission cadence safely meets the SLA?

A

Submit once every 30 hours, exactly at the SLA boundary.

B

Submit on a 4-hour cadence so worst-case completion stays within 30 hours.

✓ CORRECT

C

Submit on a 24-hour cadence and hope jobs finish faster than the window.

D

Submit continuously; the batch API auto-prioritizes to meet any SLA.



Answer B — With a 24-hour worst-case window, submitting every ~4–6 hours keeps worst-case completion under 30 hours. A 24-hour cadence risks $24+24 = 48$ hours in the worst case.

A 100-document batch returns with 6 documents failed because they exceeded the context limit. What is the most efficient recovery?

A Resubmit the entire 100-document batch from scratch — a plausible-sounding move that misreads the root cause.

B Discard the 6 failures and proceed with the 94 successes only.

C Resubmit only the 6 failed documents (by `custom_id`), chunking the oversized ones.

✓ CORRECT

D Switch the whole pipeline to the synchronous API permanently.

 Answer C — `custom_id` lets you identify and resubmit only the failed documents, applying fixes like chunking those that exceeded context limits. Re-running all 100 wastes cost.

Your extraction schema marks `supplier_tax_id` as required. For invoices that genuinely lack a tax ID, the model invents plausible-looking numbers. What schema change fixes this?

A Add a stern instruction: "Never fabricate a tax ID under any circumstances."

B Lower the temperature to 0 so the model stops guessing.

C Add a regex pattern so only well-formed tax IDs are accepted.

D Make `supplier_tax_id` optional/nullable so the model can return null when absent.

✓ CORRECT



Answer D — Marking fields nullable when source documents may not contain the info prevents the model from fabricating values just to satisfy a required field.

On validation failure you retry by appending the error and the failed extraction. It fixes formatting issues but a date field keeps failing because the date is simply not present in the document. What's true about this retry?

A Retry is ineffective here.

✓ CORRECT

B Retrying will eventually succeed if you increase the number of attempts.

C The schema must be wrong; loosen the date format and retry.

D Switch to the batch API and the retries will resolve themselves.



Answer A — Retry-with-error-feedback helps with format/structural errors, but cannot conjure information that doesn't exist in the source. Recognize absence vs. malformation. The information is absent from the source, not malformed.

Using tool_use with a strict JSON schema, you no longer get JSON syntax errors, yet some invoices have line items that don't sum to the stated total. What does this demonstrate?

A The schema is invalid and must be regenerated — an option that scales poorly as the system grows.

B Strict schemas eliminate syntax errors but not semantic errors like values that don't add up.

✓ CORRECT

C tool_choice must be set to "any" to enforce arithmetic.

D The model needs a larger context window to do addition correctly.



Answer B — Tool-use schemas guarantee well-formed JSON but not semantic correctness. Detect mismatches by extracting calculated_total alongside stated_total and flagging discrepancies.

Your extractor handles invoices with itemized tables well but returns nulls on invoices that describe charges in narrative prose. Detailed instructions haven't fixed it. What's most effective?

A Reject all narrative-format invoices as unparseable — a change that leaves the real failure mode unaddressed.

B Increase max_tokens so the model reads the whole document.

C Add few-shot examples showing correct extraction from varied formats, including prose.

✓ CORRECT

D Add a second required field to force the model to look harder.



Answer C — Few-shot examples demonstrating extraction across varied document structures (tables vs. prose) reduce empty/null extractions where instructions alone are inconsistent.

Your pipeline reports 97% overall accuracy and the team wants to drop human review. Before automating, what should you verify?

A Nothing; 97% comfortably clears any reasonable bar.

B Only the latency of the pipeline.

C Whether the batch API would raise accuracy further.

D Accuracy by document type and field.

✓ CORRECT



Answer D — A high aggregate can hide weak performance on specific document types or fields. Validate accuracy by segment before reducing human review. Aggregates can mask poor performance on specific segments.

SECTION



Tool Design

Schemas, errors, UX, distribution, and selection patterns

Q21-Q34

search_records and find_entries have near-identical one-line descriptions and overlapping inputs; the model misroutes between them constantly. What is the most effective first step?

A Expand each description with input formats, example queries, edge cases, and when to use it vs. the other.

✓ CORRECT

B Add 8 few-shot examples of correct routing to the system prompt.

C Build a keyword-based pre-router that picks the tool before the model sees the request.

D Merge both into one mega-tool that decides internally — a step that appears safe but reduces observability.



Answer A — Tool descriptions are the primary mechanism for tool selection. Differentiating them (inputs, examples, boundaries) is the low-effort, high-leverage first fix when descriptions are minimal.

A specialized synthesis subagent has access to all 18 tools in your system and frequently misuses tools meant for other roles. What principle should guide the fix?

A Give every agent all tools so none is ever blocked.

B Scope each agent's tool set to its role.

✓ CORRECT

C Rename the tools so they sort in a more logical order.

D Force tool_choice: "any" so the agent always calls something.



Answer B — Giving an agent far more tools than it needs (18 vs. 4–5) increases decision complexity and misuse. Scope tools to each role, with limited cross-role tools for high-frequency needs. Too many tools degrades selection reliability.

Your extraction pipeline must always run `extract_metadata` before any enrichment tool. How do you guarantee the metadata tool fires first?

A Set `tool_choice`: "auto" and hope the model picks it first — a tactic that trades long-term reliability for short-term ease.

B List `extract_metadata` first in the tools array and rely on ordering.

C Set `tool_choice` to forced selection { "type": "tool", "name": "extract_metadata" } for the first turn.

✓ CORRECT

D Put "always call `extract_metadata` first" in the description of every other tool.



Answer C — Forced tool selection guarantees a specific tool runs first; subsequent steps are processed in follow-up turns. "auto" only permits, never guarantees.

For a classification step, the model sometimes replies with conversational text instead of calling your classify tool. What setting forces a tool call without dictating which?

A `tool_choice: "auto"`

B `tool_choice: "none"`

C Raise `max_tokens` so the model has room to call a tool.

D `tool_choice: "any"`

✓ CORRECT



Answer D — "any" requires the model to call some tool (its choice), guaranteeing structured output over conversational text. "auto" lets it return text instead.

Your tools all return a generic "Operation failed" string on any error. The agent retries everything, including permanent policy rejections. What should the error responses include?

A `errorCategory` (transient/validation/permission), an `isRetryable` boolean, and a human-readable description.

✓ CORRECT

B A stack trace so the agent can debug the backend — an easy-to-reach answer that doesn't survive closer scrutiny.

C The same generic message but in all-caps so the agent notices it.

D Nothing; the agent should retry every failure a fixed number of times.



Answer A — Structured error metadata (category, retryable flag, readable description) lets the agent make correct recovery decisions and avoid wasted retries on non-retryable errors.

A refund exceeds the customer's eligibility window. How should `process_refund` signal this so the agent communicates appropriately rather than retrying?

- A Return a transient error so the agent retries until it succeeds.
- B Return `retriable: false` with a customer-friendly explanation of the policy.**
- C Throw an unhandled exception to halt the conversation — a fix that addresses configuration when the issue is design.
- D Return an empty success so the agent moves on quietly.

✓ CORRECT



Answer B — Business-rule violations should carry `retriable: false` plus a customer-friendly explanation, so the agent stops retrying and explains the outcome appropriately.

An `inventory_lookup` that finds no matching SKU currently returns the same `isError` response as a database timeout. Why is conflating these harmful?

A

It isn't harmful; both are just failures the agent should retry.

B

Empty results should always trigger escalation to a human.

C

A valid empty result (no matches) is a successful query; a timeout needs a retry decision.

✓ CORRECT

D

Timeouts should be silently suppressed as empty results — a choice that adds effort without changing the outcome.



Answer C — Distinguish access failures (needing retry) from valid empty results (successful queries with no matches). Treating them identically causes wrong recovery behavior. Conflating them misleads recovery.

A single `analyze_document` tool is asked to summarize, extract data points, and verify claims, and the model picks the wrong behavior unpredictably. What's the recommended redesign?

- A** Add a mode string parameter and keep one tool — an approach that the production logs would later contradict.
- B** Force `tool_choice`: "any" so the model always picks `analyze_document`.
- C** Remove the tool and have the model do everything in plain text — a move that confuses tool availability with tool ordering.

D Split it into purpose-specific tools (`extract_data_points`, `summarize_content`, `verify_claim_against_source`) with defined I/O.

✓ CORRECT



Answer D — Splitting a generic tool into purpose-specific tools with clear input/output contracts removes the ambiguity that causes unpredictable behavior.

Despite well-written tool descriptions, the model over-selects a summarize tool. You discover the system prompt repeatedly says "always summarize the customer's situation." What should you check?

A The system prompt for keyword-sensitive instructions that create unintended tool associations.

✓ CORRECT

B The model's temperature setting — an option that relies on behavior the model can't guarantee.

C Whether `tool_choice` is set to "none".

D The order of tools in the array.



Answer A — Keyword-sensitive system-prompt wording can override even good tool descriptions, creating unintended associations (e.g., "summarize" pulling the summarize tool).

A category field is an enum, but new categories appear over time and you don't want the model to fabricate or force-fit a value. What pattern keeps it extensible?

A

Remove the enum entirely and accept any free-text string.

B

Add an "other" enum value plus a detail string field for the specifics.

✓ CORRECT

C

Add every conceivable category up front so "other" is never needed.

D

Make the field required with no null option.



Answer B — An "other" + detail-string pattern (and values like "unclear" for ambiguous cases) keeps enums extensible without forcing the model to mislabel novel cases.

Your synthesis agent does many simple fact-checks (dates, names), each currently routed back through the coordinator to the web-search agent, adding latency. 85% are trivial lookups. What's the best fix?

A Give the synthesis agent every web-search tool so it never round-trips.

B Batch all verifications to the end of the synthesis pass.

C Give it a scoped `verify_fact` tool for simple lookups.

✓ CORRECT

D Pre-cache extra context for every source in case it's needed.

 Answer C — A scoped cross-role tool covers the high-frequency simple case (least privilege) while complex cases still flow through the coordinator, preserving separation of concerns. Route complex verification through the coordinator.

A generic `fetch_url` tool lets the document-analysis agent pull arbitrary pages, sometimes fetching irrelevant or unsafe URLs. What's a safer replacement?

A Keep `fetch_url` but add a prompt instruction to be careful — a path that pushes the cost onto a later step instead.

B Remove all fetching and require the user to paste content.

C Give the agent `fetch_url` plus 10 more tools so it has options.

D Replace it with `load_document` that validates the URL is an allowed document source.

✓ CORRECT



Answer D — Replacing a generic tool with a constrained alternative (`load_document` that validates document URLs) limits misuse while preserving the needed capability.

Your custom MCP search tool is more capable than the built-in Grep, yet the agent keeps defaulting to Grep. What most directly improves adoption of the MCP tool?

A Enhance the MCP tool's description to detail its capabilities and outputs vs. the built-in.

✓ CORRECT

B Disable the built-in Grep tool entirely — a change that masks the failure rather than resolving it.

C Rename the MCP tool to "grep2" so the model finds it first.

D Force tool_choice to the MCP tool on every turn.



Answer A — Detailed MCP tool descriptions explaining capabilities and outputs prevent the agent from preferring built-in tools like Grep over more capable MCP alternatives.

Reviewers keep dismissing a class of false-positive findings, but you can't tell which code constructs trigger them. What addition to the structured output enables systematic analysis?

- A A free-text comments field on each finding.
- B A detected_pattern field recording which construct triggered each finding.**
- C A timestamp on every finding — a tactic that works on the happy path but not the edge cases.
- D A confidence score and nothing else.

✓ CORRECT



Answer B — A detected_pattern field lets you systematically analyze which constructs trigger dismissed findings, so you can tune prompts for high-false-positive categories.

SECTION



MCP Design Patterns

Server scoping, resources, error protocols, and configuration

Q35-Q46

Your team needs a shared GitHub MCP server available to everyone who clones the repo, plus a personal experimental server only you use. Where do these belong?

A

Both in ~/.claude.json so they travel with each user.

B

Both in the project CLAUDE.md under an MCP heading.

C

Shared server in project-scoped .mcp.json.

✓ CORRECT**D**

Both hardcoded into each developer's shell profile.



Answer C — Project-level .mcp.json is for shared team tooling (committed to the repo); user-level ~/.claude.json is for personal/experimental servers not shared via version control. Personal server in user-scoped ~/.claude.json.

Your shared .mcp.json needs a GitHub token, but you must not commit secrets. What's the correct approach?

A Paste the literal token into .mcp.json and add the file to .gitignore.

B Store the token in CLAUDE.md so it loads with project context.

C Disable authentication for the shared server.

D Use environment variable expansion, e.g. \${GITHUB_TOKEN}, in .mcp.json.

✓ CORRECT



Answer D — Environment variable expansion (\${GITHUB_TOKEN}) lets shared .mcp.json reference credentials without committing the secret value itself.

Agents waste many exploratory tool calls just to discover what issues, docs, and schemas exist before acting. What MCP mechanism reduces this discovery overhead?

- A** Expose content catalogs (issue summaries, doc hierarchies, schemas) as MCP resources. ✓ CORRECT
- B** Add more action tools so discovery happens faster.
- C** Increase the context window so all data fits — an answer that sounds rigorous yet ignores the stated constraint.
- D** Force tool_choice: "any" to speed up exploration.



Answer A — MCP resources expose content catalogs, giving agents visibility into available data without burning tool calls on exploration. Tools are for actions; resources for catalogs.

You need a standard Jira integration plus a bespoke internal-workflow integration. How should you allocate effort?

A Build both from scratch for full control.

B Use an existing community MCP server for Jira.

✓ CORRECT

C Use community servers for both and skip the custom workflow.

D Avoid MCP entirely and call REST APIs directly from prompts.



Answer B — Prefer existing community MCP servers for standard integrations like Jira; reserve custom server effort for genuinely team-specific workflows. Reserve a custom server for the team-specific workflow.

Within MCP, how should a tool communicate that an operation failed so the agent can react?

- A Return normal content and let the agent infer failure from the wording.
- B Throw a network-level 500 so the connection drops.
- C Use the MCP `isError` flag, ideally with structured metadata about the failure.**

✓ CORRECT

- D Return an empty result and mark it successful.



Answer C — The MCP `isError` flag is the pattern for communicating tool failures back to the agent; pairing it with structured metadata enables correct recovery decisions.

A business rule says refunds above \$500 must never be auto-processed. Your prompt says so, but it occasionally slips through. What guarantees compliance?

- A Strengthen the prompt wording and add it to CLAUDE.md as well.
- B Lower the temperature so the model follows instructions exactly.
- C Add a few-shot example of declining a large refund — a choice that improves one metric while degrading another.

D Use a tool-call interception hook that blocks refunds over \$500 and redirects to escalation.

✓ CORRECT



Answer D — Prompt instructions have a non-zero failure rate. A tool-call interception hook gives deterministic enforcement, blocking policy-violating actions and redirecting them.

Three MCP tools return timestamps in different formats (Unix epoch, ISO 8601, and a numeric status code). You want the model to see them uniformly. Which hook handles this?

A A PostToolUse hook that normalizes heterogeneous tool results before the model processes them.

✓ CORRECT

B A PreToolUse hook that rewrites the request.

C A system-prompt instruction to mentally convert formats.

D tool_choice: "any" to standardize outputs — a step that introduces a new dependency for little gain.



Answer A — PostToolUse hooks intercept tool results for transformation/normalization (e.g., unifying timestamp formats) before the model reads them.

In 12% of cases the agent calls `process_refund` without ever verifying identity via `get_customer`, causing misapplied refunds. What most effectively enforces ordering?

- A A system-prompt line stating verification is mandatory.
- B A programmatic prerequisite that blocks `process_refund` until `get_customer` returns a verified ID.**

✓ CORRECT

- C Few-shot examples of always verifying first — a tempting shortcut that skips the underlying fix.
- D A routing classifier that enables tools per request type.



Answer B — Critical ordering for financial operations needs programmatic enforcement (a prerequisite gate), not probabilistic prompt-based guidance. The issue is ordering, not tool availability.

A developer asks whether tools from multiple configured MCP servers must be enabled one at a time per session. What's accurate?

A Only one MCP server can be active per session — an approach that treats the symptom rather than the cause.

B Project and user servers are mutually exclusive.

C Tools from all configured MCP servers are discovered at connection time and available simultaneously.

✓ CORRECT

D MCP tools must be manually imported in each prompt.



Answer C — All configured MCP servers' tools are discovered at connection time and are available to the agent at the same time.

A subagent hits a transient timeout calling an MCP tool. What's the recommended error-propagation behavior toward the coordinator?

A Immediately propagate every error to the coordinator and stop.

B Suppress the error and return an empty success.

C Terminate the entire workflow on the first failure.

D Attempt local recovery for transient failures.

✓ CORRECT



Answer D — Subagents should recover locally from transient failures and propagate to the coordinator only what they can't resolve, including attempted actions and partial results. Propagate only unresolved errors with what was attempted and partial results.

One source was unavailable, so part of the research couldn't be supported. How should the synthesis subagent represent this in its output?

A Annotate coverage: mark which findings are well-supported vs. which areas have gaps from unavailable sources.

✓ CORRECT

B Silently omit the unsupported area to keep the report clean.

C Fabricate plausible findings so no gap is visible — a choice that looks simpler but shifts the problem elsewhere.

D Fail the whole synthesis because one source was missing.



Answer A — Structuring synthesis output with coverage annotations distinguishes well-supported findings from gaps caused by unavailable sources, supporting honest downstream decisions.

You want agents to read your database schema reference frequently but never to mutate it. Should the schema reference be a tool or a resource?

A

A tool, because everything an agent reads must be a tool.

B

A resource, since it's a read-only content catalog rather than an action.

✓ CORRECT

C

Neither; paste the schema into every prompt manually.

D

A tool with tool_choice forced on each turn — an option that holds only under assumptions this scenario breaks.



Answer B — Resources expose read-only content catalogs (like schemas); tools are for actions. A schema reference fits the resource model and reduces exploratory tool calls.

SECTION



Agentic Systems & Code Agents

Orchestration, codebase exploration, and session design

Q47-Q52

Asked to research "the impact of automation on the workforce," the coordinator decomposes it into "factory robots," "warehouse robots," and "assembly-line robots." The final report misses white-collar, gig, and service-sector impacts. Root cause?

- A The synthesis agent failed to detect coverage gaps.
- B The web-search agent's queries were malformed — a quick fix that tends to resurface the same failure later.
- C The coordinator's task decomposition was too narrow, omitting whole relevant domains.**
- D The document-analysis agent over-filtered sources.

✓ CORRECT



Answer C — When subagents execute correctly but whole domains are missing, the root cause is overly narrow decomposition by the coordinator. The downstream agents worked within the scope they were given.

Your coordinator spawns three independent research subagents but does so across three separate turns, making them effectively sequential. How do you run them in parallel?

A Increase `max_tokens` so all three fit in one turn's output.

B Give each subagent `tool_choice: "any"`.

C Resume each subagent with `--resume` to overlap them.

D Emit multiple `Task` tool calls in a single coordinator response.

✓ CORRECT



Answer D — Parallel subagents are spawned by emitting multiple `Task` tool calls in a single response, rather than issuing them across separate turns.

Your coordinator can't spawn any subagents — invoking the Task tool silently does nothing. What's the most likely configuration issue?

A `allowedTools` for the coordinator doesn't include "Task".

✓ CORRECT

B The subagents lack system prompts.

C `tool_choice` is set to "auto" instead of "any".

D The subagents inherit the coordinator's context automatically.



Answer A — The Task tool is the mechanism for spawning subagents, and the coordinator's `allowedTools` must include "Task" for delegation to work.

A synthesis subagent produces shallow output because it doesn't seem to have the web-search and document findings. The coordinator assumed the subagent would inherit them. What's the fix?

A Subagents auto-inherit parent context, so just retry.

B Include the complete prior findings directly in the subagent's prompt.

✓ CORRECT

C Give the synthesis subagent the web-search tools so it can refetch.

D Lower the temperature of the synthesis subagent.



Answer B — Subagents operate with isolated context and do not inherit parent history. Required findings must be passed explicitly in the subagent's prompt.

Asked to understand an unfamiliar 2,000-file codebase, the agent tries to Read every file up front and exhausts context. What's the recommended exploration strategy?

A Read all files into context first, then reason once — a workaround that adds complexity without improving reliability.

B Glob every file path and paste the list as the answer.

C Use Grep to find entry points, then Read to follow imports and trace flows incrementally.

✓ CORRECT

D Switch to a model with a bigger context window and read everything.



Answer C — Build understanding incrementally: Grep for entry points, then Read to follow imports/trace flows, rather than reading all files upfront and exhausting context.

A ticket says "add a null-check to the `parseDate()` function so empty strings don't crash the import." Scope is a single, well-understood change. Which mode fits?

- A Plan mode, to explore the whole codebase first.
- B Plan mode, because any code change is inherently architectural.
- C Neither; require a human to write it.

D Direct execution, since it's a simple, well-scoped change.

✓ CORRECT



Answer D — Direct execution suits simple, well-scoped changes (like adding one validation check). Plan mode is for large-scale, multi-file, architectural work.

SECTION



AI Fluency Framework

The 4Ds model, teaching approaches, and assessment methods

Q53-Q66

The AI Fluency framework is built around four core competencies. Which set names them?

A Delegation, Description, Discernment, Diligence

✓ CORRECT

B Design, Develop, Deploy, Document

C Define, Distribute, Debug, Deliver

D Discover, Draft, Decide, Deliver — a route that optimizes effort over correctness here.



Answer A — The 4Ds of AI Fluency are Delegation, Description, Discernment, and Diligence.

A learner decides which parts of a task to hand to an AI and which to keep, and how to share the work effectively. Which of the 4Ds is this?

A

Description

B

Delegation

✓ CORRECT

C

Discernment

D

Diligence



Answer B — Delegation is about deciding what to entrust to AI vs. yourself and how to divide the work — problem, process, and platform awareness.

Crafting clear, well-structured prompts so the AI understands the task, context, and desired output maps to which competency?

A

Delegation

B

Discernment

C

Description

✓ CORRECT

D

Diligence



Answer C — Description is communicating effectively with AI: setting context, specifying the request, and shaping the interaction so the system can do its part.

Evaluating AI outputs critically — judging quality, spotting errors, and assessing whether a response actually meets the need — is which competency?

A Delegation

B Description

C Diligence

D Discernment

✓ CORRECT



Answer D — Discernment is the critical evaluation of AI products, processes, and behavior — assessing quality and catching mistakes rather than accepting output at face value.

Using AI responsibly — being transparent about its use, considering impacts, and taking accountability for outcomes — corresponds to which D?

A Diligence

✓ CORRECT

B Discernment

C Description

D Delegation



Answer A — Diligence covers responsible, transparent, and accountable engagement with AI: creation, transparency, and deployment done thoughtfully.

A student must decide whether using AI for a graded reflection essay is appropriate and disclose it if they do. Which two competencies are most directly engaged?

A

Description and Discernment

B

Delegation and Diligence

✓ CORRECT

C

Discernment and Description

D

Delegation and Description



Answer B — Deciding whether to use AI for a task is Delegation; disclosing that use and owning the outcome is Diligence.

An analyst writes a prompt, reviews the output, refines the prompt, reviews again, iterating toward a better result. Which paired competencies form this loop?

A

Delegation and Diligence

B

Discernment and Diligence

C

Description and Discernment

✓ CORRECT

D

Delegation and Description



Answer C — Iteratively crafting prompts (Description) and critically evaluating the results (Discernment) forms the core refinement loop.

An instructor grades only the final AI-assisted deliverable against a rubric, ignoring how the student worked with the AI. What kind of assessment is this?

A Process-based assessment

B Reflection-based assessment

C Peer-review assessment

D Outcome-based assessment

✓ CORRECT



Answer D — Outcome-based assessment judges the final product. It can miss how AI was used, which process- and reflection-based methods are designed to capture.

To evaluate how well students collaborate with AI, an instructor examines their prompt iterations, decisions about what to delegate, and how they checked outputs. This is best described as:

A Process-based assessment

✓ CORRECT

B Outcome-based assessment

C A timed multiple-choice quiz

D Purely automated grading



Answer A — Process-based assessment looks at how learners work with AI — delegation choices, prompting iterations, and evaluation steps — not just the final artifact.

Students submit a short write-up explaining why they delegated certain tasks to AI, how they verified the results, and what they'd do differently. Which assessment type is this?

A Outcome-based assessment

B Reflection-based assessment

✓ CORRECT

C Process-based assessment only

D Automated rubric scoring



Answer B — Reflection-based assessment asks learners to articulate their reasoning and self-evaluate their AI collaboration, surfacing metacognition the product alone won't show.

Why are realistic, open-ended scenarios valuable when teaching AI fluency rather than only definitional quizzes?

A They are faster to grade than any other format — a plausible-sounding move that misreads the root cause.

B They eliminate the need for any human evaluation.

C They require learners to exercise judgment across the 4Ds in context, not just recall terms.

✓ CORRECT

D They guarantee every student reaches the same answer.



Answer C — Scenario-based learning makes students apply Delegation, Description, Discernment, and Diligence together in realistic situations, beyond rote definition recall.

An educator wants AI fluency woven into existing courses rather than taught as one isolated module. What's the strongest rationale?

- A** It reduces total teaching hours to zero.
- B** It removes the need for any assessment — a change that leaves the real failure mode unaddressed.
- C** It ensures students never use AI — an option that scales poorly as the system grows.

D Embedding fluency in authentic disciplinary tasks builds transferable, contextual judgment.

✓ CORRECT



Answer D — Integrating AI fluency into disciplinary work develops contextual, transferable judgment, which a single standalone module is less able to achieve.

A learner notices an AI-written summary cites a statistic that doesn't appear in the source document and flags it. Which competency are they demonstrating?

A Discernment

✓ CORRECT

B Delegation

C Description

D Diligence

 Answer A — Catching an unsupported or fabricated claim is the critical-evaluation work of Discernment.

An instructor wants assessment that reflects both the quality of AI-assisted work and the student's collaboration skill. What's the best approach?

- A** Use outcome-based grading exclusively for objectivity.
- B** Combine outcome, process, and reflection methods so both product and collaboration are captured.

✓ CORRECT

- C** Use reflection-based grading exclusively to save time.
- D** Forbid AI use so assessment stays simple — a step that appears safe but reduces observability.



Answer B — A blend of outcome-, process-, and reflection-based assessment captures both the deliverable's quality and how effectively the learner worked with AI.

W R A P - U P

Summary



Context & Escalation 18%

Stateless model, case-facts blocks, lost-in-the-middle, and when to escalate.



Batch API & Extraction 12%

50% cost vs 24h window, custom_id recovery, nullable schemas, retry limits.



Tool Design 21%

Descriptions drive selection; scope per role; typed errors; forced/any tool_choice.



MCP Design Patterns 18%

Project vs user scope, env-var secrets, resources vs tools, hooks for guarantees.



Agentic Systems 12%

Coordinator decomposition, parallel Task spawns, explicit context, crash recovery.



AI Fluency 19%

The 4Ds — Delegation, Description, Discernment, Diligence — and assessment types.

REMEMBER THIS

Key Takeaways



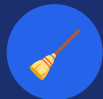
Hard rules need hard enforcement

When a question says "guarantee," "must," or "zero failures," choose programmatic hooks/prerequisite gates — not prompt wording.



Structure your handoffs & errors

Typed error metadata (errorCategory, isRetryable) and compact escalation summaries beat raw transcripts and generic failures.



Practice context hygiene

Trim verbose tool outputs, keep a persistent case-facts block, and counter lost-in-the-middle with summaries + section headers.



Escalate on the right triggers

Explicit human request, policy gap/silence, or no meaningful progress — not sentiment or self-reported confidence.



Tool selection lives in descriptions

Differentiate similar tools, scope each agent to its role, and prefer the proportionate first fix over new ML infrastructure.



Match the API to the latency

Batch API for overnight/latency-tolerant jobs; synchronous API for blocking, interactive, or multi-turn tool-calling work.

BEFORE YOU SIT

Exam Tips

- 1 Read the stem's exact ask** — Distinguish "most effective FIRST step" from "BEST/MOST effective" — the proportionate fix often wins over the maximal one.
- 2 Prefer determinism on "must"** — If a rule must hold every time (money, identity, ordering), pick hooks/gates over prompt instructions, which fail probabilistically.
- 3 Find the real root cause** — In multi-agent failures, blame the agent that made the decision (often the coordinator), not downstream agents working within their scope.
- 4 Reject over-engineering** — When prompt/description tweaks haven't been tried, a new classifier or ML pipeline is usually the wrong, heavier answer.
- 5 Classify errors precisely** — Separate transient vs validation vs business vs permission, and access failures vs valid empty results, before choosing recovery.
- 6 Let stop_reason drive loops** — Continue on "tool_use", stop on "end_turn" — never terminate on text phrases or arbitrary iteration caps.
- 7 Ignore answer length & letter** — Correct options here are randomized in position and length — decide on the concept, not the longest or most-qualified-sounding choice.